

Analyze the two GenAI programs. The analysis must include:

a. Name of two GenAI programs

ChatGPT and Gemini

b. How you accessed them (e.g., Google query, CoPilot on my computer)

I accessed both through Google Chrome on my laptop.

c. Prompt used. (You must use the same prompt for both GenAIs.):

create a c++ code that does this: The program will prioritize emails for a busy company CEO.

- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create objects from scratch. Do not include pre-existing heap modules.

• Test File format:

Command Description

EMAIL <sender category>,

<subject line>,

<date>

The emails in the CEO's Inbox should be placed in queue

based on their sender category and date. The sender

categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next

- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest

priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

d. Generated code from both GenAIs.

ChatGPT:

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <sstream>
```

```
#include <vector>
```

```
#include <string>
```

```
using namespace std;
```

```
// -----
```

```
// Email class
```

```
// -----
```

```
class Email {
```

```
private:
```

```
    string sender;
```

```
    string subject;
```

```
    string date;
```

```
public:
```

```
    Email() {}
```

```
    Email(string sender, string subject, string date) {
```

```

        this->sender = sender;

        this->subject = subject;

        this->date = date;
    }

    string getSender() const {
        return sender;
    }

    string getSubject() const {
        return subject;
    }

    string getDate() const {
        return date;
    }
};

// -----
// MaxHeap class
// -----

class MaxHeap {
private:
    vector<Email> heap;

    // Return the priority number for a sender category.

```

```

// Larger number = higher priority.
int senderPriority(const string& sender) const {
    if (sender == "Boss")
        return 5;
    else if (sender == "Subordinate")
        return 4;
    else if (sender == "Peer")
        return 3;
    else if (sender == "ImportantPerson")
        return 2;
    else
        return 1; // OtherPerson
}

```

```

// Convert MM-DD-YYYY into YYYYMMDD.
// This makes dates easy to compare numerically.
int dateValue(const string& date) const {
    int month = stoi(date.substr(0, 2));
    int day = stoi(date.substr(3, 2));
    int year = stoi(date.substr(6, 4));

    return year * 10000 + month * 100 + day;
}

```

```

// Returns true if email1 has higher priority than email2.
bool higherPriority(const Email& email1, const Email& email2) const {

```

```

int priority1 = senderPriority(email1.getSender());
int priority2 = senderPriority(email2.getSender());

// First compare sender category.
if (priority1 != priority2) {
    return priority1 > priority2;
}

// If sender categories are the same,
// the newest email has higher priority.
int date1 = dateValue(email1.getDate());
int date2 = dateValue(email2.getDate());

return date1 > date2;
}

// Move an element upward in the heap.
void heapifyUp(int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;

        if (higherPriority(heap[index], heap[parent])) {
            swap(heap[index], heap[parent]);
            index = parent;
        }
        else {

```

```
        break;
    }
}
}
```

// Move an element downward in the heap.

```
void heapifyDown(int index) {
    int size = heap.size();

    while (true) {
        int leftChild = 2 * index + 1;
        int rightChild = 2 * index + 2;
        int largest = index;

        if (leftChild < size &&
            higherPriority(heap[leftChild], heap[largest])) {
            largest = leftChild;
        }

        if (rightChild < size &&
            higherPriority(heap[rightChild], heap[largest])) {
            largest = rightChild;
        }

        if (largest != index) {
            swap(heap[index], heap[largest]);
        }
    }
}
```

```
        index = largest;
    }
    else {
        break;
    }
}
}
```

public:

// Add an email to the MaxHeap.

```
void insert(const Email& email) {
```

```
    heap.push_back(email);
```

// Restore heap property.

```
    heapifyUp(heap.size() - 1);
```

```
}
```

// Return the highest-priority email without removing it.

```
Email getMax() const {
```

```
    return heap[0];
```

```
}
```

// Remove the highest-priority email.

```
void removeMax() {
```

```
    if (heap.empty()) {
```

```

        return;
    }

    // Move last element to root.
    heap[0] = heap.back();
    heap.pop_back();

    // Restore heap property.
    if (!heap.empty()) {
        heapifyDown(0);
    }
}

// Check whether heap is empty.
bool isEmpty() const {
    return heap.empty();
}

// Return number of emails.
int size() const {
    return heap.size();
}

};

// -----
// Print the highest-priority email

```

```

// -----
void printNextEmail(const MaxHeap& emailQueue) {
    if (emailQueue.isEmpty()) {
        return;
    }

    Email email = emailQueue.getMax();

    cout << "Next email:" << endl;
    cout << "\tSender: " << email.getSender() << endl;
    cout << "\tSubject: " << email.getSubject() << endl;
    cout << "\tDate: " << email.getDate() << endl;
    cout << endl;
}

// -----
// Main
// -----

int main(int argc, char* argv[]) {

    // The test file is supplied as a command-line argument.
    if (argc < 2) {
        cerr << "Error: Please provide a test file." << endl;
        return 1;
    }
}

```

```
ifstream inputFile(argv[1]);
```

```
if (!inputFile) {
```

```
    cerr << "Error: Could not open file." << endl;
```

```
    return 1;
```

```
}
```

```
MaxHeap emailQueue;
```

```
string line;
```

```
while (getline(inputFile, line)) {
```

```
    // Skip empty lines.
```

```
    if (line.empty()) {
```

```
        continue;
```

```
    }
```

```
    // -----
```

```
    // EMAIL command
```

```
    // -----
```

```
    if (line.substr(0, 5) == "EMAIL") {
```

```
        // Remove "EMAIL " from the beginning.
```

```
        string emailData = line.substr(6);
```

```

// Find the commas separating the fields.
size_t firstComma = emailData.find(',');
size_t secondComma = emailData.find(',', firstComma + 1);

string sender =
    emailData.substr(0, firstComma);

string subject =
    emailData.substr(
        firstComma + 1,
        secondComma - firstComma - 1
    );

string date =
    emailData.substr(secondComma + 1);

Email newEmail(sender, subject, date);

emailQueue.insert(newEmail);
}

// -----
// NEXT command
// -----
else if (line == "NEXT") {

```

```

// NEXT does NOT remove the email.

// Therefore another NEXT displays the same email.
if (!emailQueue.isEmpty()) {
    printNextEmail(emailQueue);
}
}

```

```

// -----
// READ command
// -----
else if (line == "READ") {

```

```

// READ removes the highest-priority email.
// It does not display anything.
if (!emailQueue.isEmpty()) {
    emailQueue.removeMax();
}
}

```

```

// -----
// COUNT command
// -----
else if (line == "COUNT") {

```

```

    cout << "There are "
        << emailQueue.size()

```

```

        << " emails to read.\n"
        << endl;
    }
}

inputFile.close();

return 0;
}

```

Gemini:

```

#include <iostream>
#include <string>
#include <sstream>
#include <vector>
#include <iomanip>

// Represents an Email record
struct Email {
    std::string senderCategory;
    std::string subject;
    std::string dateStr; // Original MM-DD-YYYY

    // Converted fields for priority comparison
    int categoryPriority;

```

```
int year;  
int month;  
int day;
```

```
// Determine category priority weight (Higher number = Higher priority)
```

```
static int getCategoryPriority(const std::string& cat) {  
    if (cat == "Boss") return 5;  
    if (cat == "Subordinate") return 4;  
    if (cat == "Peer") return 3;  
    if (cat == "ImportantPerson") return 2;  
    return 1; // OtherPerson  
}
```

```
// Parse date string MM-DD-YYYY into integers
```

```
void parseDate(const std::string& d) {  
    dateStr = d;  
    std::stringstream ss(d);  
    char dash1, dash2;  
    ss >> month >> dash1 >> day >> dash2 >> year;  
}
```

```
// Custom comparison for MaxHeap (operator> defines higher priority)
```

```
bool operator>(const Email& other) const {  
    // Primary priority: Sender Category  
    if (categoryPriority != other.categoryPriority) {  
        return categoryPriority > other.categoryPriority;  
    }
```

```
}
```

```
// Secondary priority: Newest Email First (Year -> Month -> Day)
```

```
if (year != other.year) {
```

```
    return year > other.year;
```

```
}
```

```
if (month != other.month) {
```

```
    return month > other.month;
```

```
}
```

```
return day > other.day;
```

```
}
```

```
bool operator<(const Email& other) const {
```

```
    return !(*this > other || (categoryPriority == other.categoryPriority &&  
        year == other.year && month == other.month && day == other.day));
```

```
}
```

```
};
```

```
// Custom dynamic array/list implementation of MaxHeap built from scratch
```

```
class MaxHeap {
```

```
private:
```

```
    std::vector<Email> heap;
```

```
void heapifyUp(int index) {
```

```
    while (index > 0) {
```

```
        int parentIndex = (index - 1) / 2;
```

```

    if (heap[index] > heap[parentIndex]) {
        std::swap(heap[index], heap[parentIndex]);
        index = parentIndex;
    } else {
        break;
    }
}
}

```

```

void heapifyDown(int index) {
    int size = heap.size();
    while (true) {
        int maxIndex = index;
        int leftChild = 2 * index + 1;
        int rightChild = 2 * index + 2;

        if (leftChild < size && heap[leftChild] > heap[maxIndex]) {
            maxIndex = leftChild;
        }

        if (rightChild < size && heap[rightChild] > heap[maxIndex]) {
            maxIndex = rightChild;
        }

        if (maxIndex != index) {
            std::swap(heap[index], heap[maxIndex]);
            index = maxIndex;
        }
    }
}

```

```
        } else {  
            break;  
        }  
    }  
}
```

public:

```
void insert(const Email& email) {  
    heap.push_back(email);  
    heapifyUp(heap.size() - 1);  
}
```

```
void removeMax() {  
    if (heap.empty()) return;  
    heap[0] = heap.back();  
    heap.pop_back();  
    heapifyDown(0);  
}
```

```
Email getMax() const {  
    if (!heap.empty()) {  
        return heap[0];  
    }  
    return Email();  
}
```

```

bool isEmpty() const {
    return heap.empty();
}

int size() const {
    return heap.size();
}
};

// Trim helper functions to clean up space padding from split strings
std::string trim(const std::string& str) {
    size_t start = str.find_first_not_of(" \t");
    if (start == std::string::npos) return "";
    size_t end = str.find_last_not_of(" \t");
    return str.substr(start, end - start + 1);
}

int main() {
    MaxHeap emailQueue;

    std::string line;

    while (std::getline(std::cin, line)) {
        if (line.empty()) continue;

        if (line.rfind("EMAIL ", 0) == 0) {
            // Command format: EMAIL <sender category>, <subject line>, <date>

```

```

std::string data = line.substr(6);
std::stringstream ss(data);

std::string category, subject, dateStr;
std::getline(ss, category, ',');
std::getline(ss, subject, ',');
std::getline(ss, dateStr, ',');

Email email;
email.senderCategory = trim(category);
email.subject = trim(subject);
email.categoryPriority = Email::getCategoryPriority(email.senderCategory);
email.parseDate(trim(dateStr));

emailQueue.insert(email);
}
else if (line == "COUNT") {
    std::cout << "There are " << emailQueue.size() << " emails to read." << std::endl;
    std::cout << std::endl;
}
else if (line == "NEXT") {
    if (!emailQueue.isEmpty()) {
        Email top = emailQueue.getMax();
        std::cout << "Next email:" << std::endl;
        std::cout << "\tSender: " << top.senderCategory << std::endl;
        std::cout << "\tSubject: " << top.subject << std::endl;
    }
}

```

```

        std::cout << "\tDate: " << top.dateStr << std::endl;

        std::cout << std::endl;
    }
}

else if (line == "READ") {
    if (!emailQueue.isEmpty()) {
        emailQueue.removeMax();
    }
}
}

return 0;
}

```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

a) Correctness

Both Gemini and ChatGPT don’t tell the user if they enter an invalid date, however ChatGPT will throw an exception, which is not handled, and Gemini will store the incorrect date. ChatGPT should also be edited to be able to prioritize two emails that have the same date based off of what order they came in.

b) Execution time

When evaluating the date, Gemini only looks at it once and then does the comparison with the value assigned to the email whereas in ChatGPT, every time it

compares two email date values it is getting the date again twice and calls dateValue. So, while both are still a time complexity of $O(\log n)$, the second takes more work to do the comparison part, making Gemini the better code in terms of execution time.

c) Space Complexity

Both codes have a $O(n)$ space complexity, although ChatGPT only stores the sender subject and date whereas Gemini also stores the category priority, year, month, and day. ChatGPT does have fewer initialized variables, making it better for space complexity, however the drawback of this is the previously mentioned extra work the CPU has to go through for the comparisons.

d) Maintainability

ChatGPT has a few more comments than Gemini which makes it generally more maintainable; however, the Gemini code has better organization when assigning responsibilities between the email class and the heap class, which I believe is more critical than comments in the code, making Gemini overall more maintainable.

f. Select one to use as the basis for this assignment and justify why you picked it.

I am going to use Gemini because it has better overall correctness, execution time and maintainability. While ChatGPT may have better space complexity, I believe it hurts the other categories in a way that makes it overall not better when looking at the total comparison between the two. Therefore, I believe that Gemini will be the better code for me to edit to make it meet the requirements I need it to.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

a) Correctness

The cases where invalid dates are entered will now tell the user that the date was invalid and won't add that email to the heap. Also made it so it will tell the user if they try to do next or read when there are no emails in the heap. Finally, changed where if

EMAIL is on the input file it will not treat it as an email if it does not have the email details.

b) Execution time

In order to fix the correctness while handling different inputs, the execution time increased slightly however it still has an overall time complexity of $O(\log n)$. I would consider this slight increase in execution time necessary because correctness should be the top priority.

c) Space Complexity

Space complexity also unfortunately increased slightly with the correctness fixing; however, it is still $O(n)$ complexity and as optimal as it can be with the level of maintainability that it is at. I would also consider this slight increase in space complexity necessary because correctness should be the top priority.

d) Maintainability

Added comments to every line as well as a prologue comment and comments explaining large blocks of code. This will allow for better readability of the code and improve comprehension upon a reader's first glance, making the code much more maintainable.